

NESSAI 采样原理、Triangle-BBH 官方路线与本仓库实现说明

任务五子任务二：Taiji MBHB 参数估计技术笔记

雷畅

代码仓库: [task5-lisa-taiji](#)

2026 年 7 月

摘要

本文档面向任务五子任务二的技术解释，重点说明 NESSAI 嵌套采样的工作原理、官方 Triangle-BBH Example 4 的 CPU 路线、本仓库 [task5-lisa-taiji](#) 的 GPU 杂频似然 + 向量化 NESSAI 路线、两条路线 posterior distribution 图存在差异的代码层原因，以及任务要求的 5-day 窗口 ($t_c - 4 \text{ day}$ 到 $t_c + 1 \text{ day}$) 在代码中的具体实现方式。文档将论文中的算法机制、官方仓库定位和本仓库函数/变量逐项对应，避免把 NESSAI 理解为普通 MCMC 或神经网络直接回归参数。核心结论是：本仓库没有改变贝叶斯推断目标，而是将 Triangle-BBH 的 GPU heterodyned likelihood 包装成 `nessai.model.Model`，使 NESSAI 能用批量 GPU 似然完成 full nested sampling; posterior 图与官方 CPU 示例不同，主要源于先验范围、fiducial 分支、似然近似和参数坐标空间不同，而不是采样失败。

目录

1	需要回答的核心问题	3
2	文献中的 NESSAI 算法原理	3
2.1	从 nested sampling 到 constrained prior	3
2.2	NESSAI 的 normalising-flow proposal	3
2.3	诊断: insertion index、边界和 P-P 检验	4
2.4	近两年相关应用	4
3	官方 Triangle-BBH Example 4 的 CPU 路线	4
3.1	官方仓库定位	4
3.2	CPU 路线的逻辑链	5
4	本仓库 GPU 杂频似然 + NESSAI 的实现	5
4.1	为什么要改造官方路线	5
4.2	NESSAI 参数空间: 内部坐标与物理坐标	5
4.3	GPU heterodyned likelihood 的构造	6
4.4	Fisher 局部先验盒	7
4.5	自定义 NESSAI Model	7

目录	2
4.6 FlowSampler 调用	8
5 官方 CPU posterior 与本仓库 posterior 差异的原因	8
5.1 不是采样失败，而是统计问题不同	8
5.2 先验范围差异	8
5.3 F-stat 搜索中心落在 reflected branch	9
5.4 为什么不能简单开启 wide-sky	9
6 5-day 窗口期的具体实现逻辑	9
6.1 窗口定义	9
6.2 数据切片与 FFT/PSD 重建	10
6.3 5-day 窗口对象如何进入后续流程	10
6.4 5-day 实际运行结果	11
7 从代码角度总结整个采样链	11
8 结论	12

1 需要回答的核心问题

围绕子任务二，导师最可能追问四类问题：

1. NESSAI 到底如何采样？它与普通 MCMC、dynesty 或直接神经网络回归有什么区别？
2. 官方 [Triangle-BBH Example 4](#) 的 CPU 路线如何组织数据、似然和采样？
3. 本仓库为什么使用 GPU heterodyned likelihood + vectorised NESSAI？具体定义了哪些函数和变量？
4. 为什么官方 CPU posterior 图和本仓库 posterior 图不同？5-day 窗口又是怎样真正进入代码的？

本文档的回答方式是从统计目标出发，再落到代码实现。NESSAI 的目标仍然是贝叶斯后验和证据：

$$p(\theta|d, H) = \frac{\mathcal{L}(d|\theta, H)\pi(\theta|H)}{Z}, \quad Z = \int \mathcal{L}(d|\theta, H)\pi(\theta|H) d\theta.$$

其中 θ 是 MBHB 参数， d 是频域 A/E 通道数据， $\mathcal{L}$ 是高斯噪声假设下的频域似然， π 是先验， Z 是 Bayesian evidence。NESSAI 的创新在于更高效地产生 nested sampling 的替换 live point，而不是改变上式。

2 文献中的 NESSAI 算法原理

2.1 从 nested sampling 到 constrained prior

标准 nested sampling 维护 N_{live} 个 live points。每一步删除似然最低的点，其似然记为 L_* ，然后从受限先验

$$\pi(\theta|\mathcal{L}(\theta) > L_*)$$

中抽取一个新点替换它。该过程逐步压缩先验体积 X ，并用被丢弃点和最终 live points 估计 evidence 和 posterior。

困难在于：高维引力波参数空间中的等似然面往往弯曲、相关、周期性强、多模态，直接从 $\mathcal{L} > L_*$ 区域采样很难。传统做法包括随机游走 MCMC、椭圆包络或动态 nested sampling；这些方法在昂贵似然下的主要成本是大量 likelihood evaluations。

2.2 NESSAI 的 normalising-flow proposal

Williams, Veitch & Messenger 提出的 NESSAI 使用 normalising flow 学习当前 live points 在等似然面内的分布 [1]。其核心步骤为：

1. 用当前 live points 训练一个可逆映射 $f: X \rightarrow Z$ ，其中 X 是采样空间， Z 是 latent Gaussian 空间。
2. 将当前 worst live point θ_* 映射到 $z_* = f(\theta_*)$ 。在 Gaussian latent 空间中， $\|z_*\|$ 给出一个近似的 n-ball 半径。
3. 在 latent n-ball 中批量采样新点 z_i ，再用 flow 逆映射 $\theta_i = f^{-1}(z_i)$ 回到参数空间。

4. 用 prior/proposal ratio 做 rejection correction, 保证样本服从原始采样先验, 而不是单纯服从 flow proposal。
5. 合格点进入 proposal pool。后续 nested sampling 从 pool 中取点替换 worst live point; 当 pool 耗尽、接受率下降或达到 retraining 条件时, 重新训练 flow。

因此, NESSAI 的神经网络部分只负责 proposal; 是否接受一个点仍然由真实 likelihood 和 nested sampling 约束决定。论文验证了 NESSAI 在 128 个模拟 compact binary coalescence 信号上的无偏性, 并与 dynesty 的 log-evidence 一致, 同时减少似然调用次数。该结论对本项目很重要, 因为 MBHB 频域响应和 TDI 似然也属于昂贵似然问题。

2.3 诊断: insertion index、边界和 P-P 检验

NESSAI 论文强调诊断的重要性。主要包括:

- **insertion index:** 新 live point 插入现有 live points 的似然排序位置应服从均匀分布。若明显偏离, 可能说明 proposal 过约束或欠约束。
- **logX-logL trace:** 检查先验体积压缩和似然增长是否平滑。
- **P-P calibration:** 需要多次注入才能验证总体无偏性。单个 TDC 注入只能报告 insertion-index、边界检查和 trace 等单次运行诊断。

本仓库保存了 NESSAI 原生诊断图, 同时输出 JSON 形式的辅助 KS 检验和 boundary fraction; 这与 NESSAI 论文建议一致。

2.4 近两年相关应用

近两年开放获取文献中, NESSAI 继续作为引力波推断中的重要 sampler 或 benchmark:

- Covas, Prix & Martins 的连续波候选 follow-up 框架使用 Bilby 包装 MCMC 或 nested sampling, 并比较 DYNESTY、NESSAI 和 PTEMCEE, 说明 NESSAI 已被用于候选后续参数估计, 而不只是方法论文的示例 [2]。
- El Gammal 等在 LISA deterministic source inference 中将 NESSAI 作为 cutting-edge nested sampler 基准, 测试 DWD、stellar-mass BBH 和 SMBHB 信号, 说明 LISA/space-based inference 的计算瓶颈与本任务高度相关 [3]。

3 官方 Triangle-BBH Example 4 的 CPU 路线

3.1 官方仓库定位

Triangle-BBH README 将该库描述为 frequency-domain BBH TDI-2.0 response template and basic analysis tools。官方说明其包含两类波形响应包装: BBHx 的 IMRPhenomD/HM CPU/GPU 路线, 以及 WF4Py 的 IMRPhenomD/HM CPU 路线。同时, Example 4 被定位为 Taiji Data Challenge II preliminary analysis example, 而不是完整挑战解法; 官方也明确指出示例未覆盖更真实噪声、信号重叠和更复杂波形 [4]。

因此, 官方 Example 4 的作用更像教学基线: 展示如何读 TDC 数据、构造 TDI 频域数据、用 F-statistics 搜索 MBHB 参数、进行 Fisher 分析和 posterior sampling。

3.2 CPU 路线的逻辑链

官方 CPU 路线可概括为：

TDI $X/Y/Z \rightarrow A/E/T$ 或 $A/E \rightarrow \text{FFT/PSD/Covariance}$
 $\rightarrow \text{WF4Py CPU response} \rightarrow \text{F-statistics search}$
 $\rightarrow \text{Fisher prior} \rightarrow \text{Bilby/NESSAI posterior}.$

本仓库保留了这条 CPU/Bilby/NESSAI 参考路径，但默认不执行：

```
RUN_CPU_NESSAI = False
OFFICIAL_SAMPLER_SETTINGS = dict(sampler="nessai", nlive=1200, stopping=0.1)
```

对应位置在 `scripts/build_subtask2_notebook.py` 的 runtime switch 区域。CPU 路线中的 likelihood wrapper 为：

```
class BilbyLikelihoodWrapper(bilby.Likelihood):
    def log_likelihood(self):
        parameter_array = ParamDict2ParamArrFref(self.parameters)
        if self.like_type == "heterodyned":
            return self.like_object.het_log_like(parameter_array=parameter_array)
        return self.like_object.full_log_like(parameter_array=parameter_array)
```

其先验由 `build_priors(search, FIM)` 构造。质量、自旋、时间和距离使用搜索点加 Fisher 误差；角度参数则比较宽：

```
reference_phase ~ Uniform(0, 2*pi)
inclination ~ Sine(0, pi)
longitude ~ Uniform(0, 2*pi), periodic
latitude ~ Cosine(-pi/2, pi/2)
psi ~ Uniform(0, pi), periodic
```

这意味着官方/CPU 风格的 posterior 图更容易展示全局结构，包括 direct/reflected 天区简并和周期参数导致的多峰形态。

4 本仓库 GPU 杂频似然 + NESSAI 的实现

4.1 为什么要改造官方路线

任务要求保留 NESSAI 采样，但直接 CPU full-frequency likelihood 对 MBHB/TDI 数据非常慢。你的仓库采用的改造是：

Triangle-BBH GPU response + heterodyned likelihood + vectorised nessai.Model.

该路线并未改变贝叶斯目标，只是让 NESSAI 每次能够批量评估 live points，且每个似然评估只在 fiducial waveform 附近使用杂频近似。实际 full run 结果显示，baseline 和 5-day 两个窗口分别约 9.44 分钟和 9.59 分钟完成。

4.2 NESSAI 参数空间：内部坐标与物理坐标

本仓库的 NESSAI 不直接在所有物理参数上均匀采样，而是在 Triangle-BBH 内部参数坐标中采样：

```
GPU_NESSAI_INTERNAL_PARAM_NAMES = [
    "log_chirp_mass",
    "mass_ratio",
    "spin_1z",
    "spin_2z",
    "coalescence_time",
    "coalescence_phase",
    "log_luminosity_distance",
    "cos_inclination",
    "longitude",
    "sin_latitude",
    "psi",
]
```

输出和报告时再转回物理参数:

```
GPU_NESSAI_PHYSICAL_PARAM_NAMES = [
    "chirp_mass",
    "mass_ratio",
    "spin_1z",
    "spin_2z",
    "coalescence_time",
    "coalescence_phase",
    "luminosity_distance",
    "inclination",
    "longitude",
    "latitude",
    "psi",
]
```

这一步非常重要, 因为 `log_chirp_mass` 和 `log_luminosity_distance` 上的均匀先验, 对应物理量上的 Jeffreys-like 先验; `cos_inclination` 和 `sin_latitude` 更适合角度各向同性, 但 corner 图显示时又会转回 ι 和 β 。

4.3 GPU heterodyned likelihood 的构造

代码通过 `build_gpu_heterodyned_likelihood(gpu_search)` 构造似然:

```
Like_gpu = Likelihood(
    response_generator=FDTDI_GPU,
    frequency=gpu_search["gpu_window"]["frequency"],
    data=gpu_search["gpu_window"]["data"],
    invserse_covariance_matrix=gpu_search["gpu_window"]["inv_covariance"],
    response_parameters=gpu_search["response_kwargs_direct"],
    use_gpu=True,
)
Like_gpu.prepare_het_log_like(
    base_parameters=ParamDict2ParamArr(gpu_search["searched_parameters"])
)
```

这里 `base_parameters` 是 F-statistics 搜索得到的 fiducial 参数。heterodyned likelihood 的有效性依赖待评估参数靠近该 fiducial，因此后面的先验盒必须是 local-fisher，而不是任意 wide-sky。

4.4 Fisher 局部先验盒

NESSAI bounds 由 `build_gpu_nessai_bounds(gpu_search, FIM)` 生成。核心参数为：

```
GPU_NESSAI_PRIOR_SIGMA = 8.0
GPU_NESSAI_SKY_PRIOR_MODE = "local_fisher"
GPU_NESSAI_LOCAL_MAX_HALF_WIDTHS = {
    "coalescence_phase": 0.5,
    "inclination": 0.25,
    "longitude": 0.5,
    "latitude": 0.5,
    "psi": 0.5,
}
```

逻辑为：以 F-stat 搜索点为中心，用 $8\sigma_{\text{Fisher}}$ 构造盒子；如果某些角度参数的 Fisher 盒太宽，就用上面的最大半宽裁剪。随后将物理 bounds 用 `ParamDict2ParamArr` 转换到 Triangle-BBH 内部坐标，交给 NESSAI。

4.5 自定义 NESSAI Model

本仓库最核心的类是：

```
class TaijiGPUHeterodynedNESSAIModel(Model):
    def __init__(self, param_names, bounds, like_gpu, likelihood_chunksize=None):
        self.names = list(param_names)
        self.bounds = bounds
        self._like_gpu = like_gpu
        self.allow_vectorised = True
        self.vectorised_likelihood = True
        self.likelihood_chunksize = likelihood_chunksize
```

`log_prior` 实现内部坐标盒上的均匀先验：

```
def log_prior(self, x):
    log_p = np.full(x.size, -np.inf)
    in_bounds = self.in_bounds(x)
    width_log_norm = 0.0
    for name in self.names:
        width_log_norm += np.log(self.bounds[name][1] - self.bounds[name][0])
    log_p[in_bounds] = -width_log_norm
    return log_p
```

`log_likelihood` 将 NESSAI 的 structured array 拼成 (n_{dim}, N) 矩阵，并调用 Triangle-BBH 的批量 GPU 杂频似然：

```
params = np.vstack([np.atleast_1d(x[name]).astype(float) for name in self.names])
ll_valid = self._like_gpu.het_log_like_vectorized(params_eval)
```

这正是本仓库相对官方 CPU 路线的关键：NESSAI 仍然负责 nested sampling，但 likelihood evaluation 变成 GPU 批量评估。

4.6 FlowSampler 调用

生产运行由 `run_gpu_nessai_sampler(gpu_search, FIM, label)` 完成：

```
fs = FlowSampler(
    model,
    output=str(output),
    nlive=GPU_NESSAI_NLIVE,
    stopping=GPU_NESSAI_STOPPING,
    seed=GPU_NESSAI_SEED,
    resume=GPU_NESSAI_RESUME,
    likelihood_chunksize=GPU_NESSAI_LIKELIHOOD_CHUNKSIZE,
    pytorch_threads=1,
)
fs.run(plot=GPU_NESSAI_PLOT_DIAGNOSTICS, save=True)
```

实际运行配置为：

参数	数值
GPU_NESSAI_RUN_MODE	full
GPU_NESSAI_NLive / full_nlive	2000
stopping	0.1
seed	1234
likelihood_chunksize	512
sky_prior_mode	local_fisher
sampler	nessai.FlowSampler
likelihood	Triangle_BBH GPU heterodyned het_log_like_vectorized

5 官方 CPU posterior 与本仓库 posterior 差异的原因

5.1 不是采样失败，而是统计问题不同

官方 posterior 图和本仓库 posterior 图不应被要求完全一致。主要原因是：

1. 官方 CPU 示例更接近宽先验/全局示例展示；本仓库是反射支附近的 local-fisher posterior。
2. 官方路线偏 full-frequency CPU likelihood；本仓库主线使用 fiducial 附近有效的 GPU heterodyned likelihood。
3. 官方角度先验较宽，会展示 direct/reflected 简并；本仓库为保证杂频近似有效，主动限制角度半宽。
4. 本仓库在内部参数坐标上采样，再转回物理坐标展示，视觉形状也会改变。

5.2 先验范围差异

CPU/Bilby 参考路线中，角度参数为宽先验：

$$\lambda \sim U(0, 2\pi), \quad \beta \sim \cos \beta, \quad \iota \sim \sin \iota, \quad \psi \sim U(0, \pi).$$

本仓库 GPU NESSAI 路线中, baseline 实际 Fisher 盒示例为:

```
inclination: [1.863, 1.907]
longitude:   [1.671, 2.671]
latitude:    [-1.285, -0.285]
psi:         [1.361, 2.361]
```

5-day 窗口的实际盒子也类似:

```
inclination: [1.859, 1.910]
longitude:   [1.702, 2.702]
latitude:    [-1.313, -0.313]
psi:         [1.336, 2.336]
```

因此, 本仓库 posterior 不会展示被先验盒排除的 direct branch 或其它远端天区模态。

5.3 F-stat 搜索中心落在 reflected branch

代码中 F-statistics 搜索会保存 direct 和 reflected 两套参数:

```
searched_parameters_reflected = get_reflected_parameter_dict(
    searched_params=searched_parameters,
    orbit=orbit
)
```

实际结果显示, 搜索最大值更接近 ecliptic-plane reflected branch。corner 图中橙色实线表示 direct injection, 红色虚线表示 reflected reference。因此判断外禀参数恢复时, 应当看 posterior 与红色虚线的关系, 而不是只看橙色 direct injection。

5.4 为什么不能简单开启 wide-sky

将 GPU_NESSAI_SKY_PRIOR_MODE 改成 wide_sky 表面上能覆盖更大天区, 但会破坏 heterodyned likelihood 的局部有效前提。远离 fiducial waveform 时, 模板相位差不再平滑, 杂频近似可能失真。更严谨的做法是:

1. 以 direct branch fiducial 跑一次 local-fisher NESSAI;
2. 以 reflected branch fiducial 跑一次 local-fisher NESSAI;
3. 比较两支的 $\log Z$ 、posterior、reconstruction 和 residual;
4. 若需要全局图, 再合并分支 posterior 或做 mixture analysis。

6 5-day 窗口期的具体实现逻辑

6.1 窗口定义

任务要求的 5-day 窗口不是官方 baseline 的对称窗口, 而是:

$$[t_c - 4 \text{ day}, t_c + 1 \text{ day}].$$

代码定义在 `get_window_bounds(tc_day, mode)` 中:

```
def get_window_bounds(tc_day: float, mode: str) -> tuple[float, float]:
    if mode == "official_baseline":
        return tc_day - 2.5, tc_day + 2.5
    if mode == "task_five_day":
        return tc_day - 4.0, tc_day + 1.0
    raise ValueError(f"Unknown mode: {mode}")
```

6.2 数据切片与 FFT/PSD 重建

`build_window_data(label, mode, psd_mode="before")` 是窗口构造的核心函数。它执行：

1. 从注入参数读取 t_c ;
2. 根据 `mode` 得到 `start_day` 和 `end_day`;
3. 用 `mask` 从完整 TDI 时间序列切出当前窗口;
4. 对窗口内 A/E 通道重新做 Tukey-window FFT;
5. 用窗口开始之前的数据估计 PSD;
6. 截取频段 $5 \times 10^{-5} \text{ Hz} \leq f \leq 10^{-2} \text{ Hz}$;
7. 重新构造 `CovMat` 和 `InvCovMat`。

关键代码为：

```
tc_day = get_tc_day(injected_parameters)
start_day, end_day = get_window_bounds(tc_day, mode)
mask = (full_time / DAY >= start_day) & (full_time / DAY <= end_day)
data_time = full_time[mask]
data_channels_td = full_channels_td[:, mask]
```

注意 `mask` 使用闭区间，因此 5 天、采样间隔 $dt = 10 \text{ s}$ 时会得到约

$$5 \times 86400/10 + 1 = 43201$$

个采样点。README 中记录 `baseline` 和 `5-day` 均为 43201 samples。

6.3 5-day 窗口对象如何进入后续流程

窗口对象创建处为：

```
five_day_window = build_window_data(
    "task_five_day",
    "task_five_day",
    psd_mode="before"
)
plot_window_timeseries(five_day_window, "05_five_day_timeseries.png")
plot_window_frequency(five_day_window, "06_five_day_frequency_psd.png")
```

随后该对象进入完整 GPU 路线：

```

five_day_gpu_preflight = run_gpu_preflight(five_day_window, "task_five_day")
five_day_gpu_search, five_day_gpu_FIM = load_gpu_search_from_cache(
    five_day_window, "task_five_day"
)
five_day_gpu_FIM = run_gpu_fisher_analysis(five_day_window, five_day_gpu_search)
five_day_gpu_nessai_sampler, five_day_gpu_summary = run_gpu_nessai_sampler(
    five_day_gpu_search, five_day_gpu_FIM, "task_five_day"
)

```

因此，5-day posterior 是对 5-day 数据窗口完整重跑的结果，不是 baseline posterior 的复用或改名。

6.4 5-day 实际运行结果

本仓库中 5-day full GPU NESSAI 结果为：

指标	数值
$\log Z$	594995.4055
$\sigma_{\log Z}$	0.1246
posterior samples	10879
wall time	9.589 min
insertion-index KS p	0.3567
max boundary fraction	0.00735

这些数值说明 5-day 运行本身通过了轻量 insertion-index 和 boundary-contact 检查。但必须强调：baseline 与 5-day 使用的是不同数据段，因此 $\Delta \log Z$ 不能解释为 Bayes factor。可以比较的是 posterior CI 宽度、形态、重构残差和诊断质量。

7 从代码角度总结整个采样链

本仓库子任务二的主链条可以总结为：

读取 TDC $X/Y/Z \rightarrow$ 构造 $A/E \rightarrow$ 选择窗口 $\rightarrow$ FFT/PSD/Covariance
 $\rightarrow$ GPU F-statistics search $\rightarrow$ Fisher local prior $\rightarrow$ GPU heterodyned likelihood
 $\rightarrow$ custom nessai.Model $\rightarrow$ FlowSampler full run $\rightarrow$ posterior/evidence/diagnostics.

其中最关键的函数和变量如下：

代码对象	作用
get_window_bounds	定义 official baseline 与 task 5-day 窗口边界。
build_window_data	切片数据并重建 FFT、PSD、协方差和逆协方差。
run_gpu_fstat_search	在当前窗口上运行 GPU F-statistics search。
run_gpu_fisher_analysis	用搜索点和窗口数据计算 Fisher 误差。
build_gpu_nessai_bounds	用搜索点和 Fisher 误差构造 local-fisher 先验盒。

<code>build_gpu_</code>	以搜索点为 fiducial 准备 GPU 杂频似然。
<code>heterodyned_likelihood</code>	
<code>TaijiGPUHeterodyned</code>	将 Triangle-BBH GPU 似然包装为 NESSAI model。
<code>NESSAIModel</code>	
<code>log_prior</code>	定义内部参数盒上的均匀先验。
<code>log_likelihood</code>	调用 <code>het_log_like_vectorized</code> 批量评估似然。
<code>run_gpu_nessai_sampler</code>	构造 <code>FlowSampler</code> , 运行 NESSAI 并保存结果。
<code>posterior_boundary_check</code>	检查 posterior 是否贴 Fisher 盒边界。
<code>extract_nessai_diagnostics</code>	保存 insertion-index KS 辅助诊断。

8 结论

本仓库的 NESSAI 采样路线可以概括为“统计目标不变，计算路径优化”。官方 CPU Example 4 提供了空间引力波 MBHB 参数估计的基线流程；本仓库保留了该流程的科学结构，但将昂贵的 CPU/full-frequency 似然替换为 Triangle-BBH GPU heterodyned vectorised likelihood，并通过自定义 `nessai.model.Model` 接入 `nessai.FlowSampler`。posterior 图与官方 CPU 图不同，主要是因为本仓库采用 local-fisher 反射支后验，而官方图更接近宽范围全局示例 posterior。5-day 窗口由 `get_window_bounds` 和 `build_window_data` 从数据层面重新构造，后续 F-stat、Fisher、likelihood 和 NESSAI 都使用同一个 `five_day_window` 对象，因而结果具有完整的数据处理闭环。

参考文献

- [1] M. J. Williams, J. Veitch, and C. Messenger. Nested Sampling with Normalising Flows for Gravitational-Wave Inference. [arXiv:2102.11056](#), 2021.
- [2] P. B. Covas, R. Prix, and J. Martins. A new framework to follow up candidates from continuous gravitational-wave searches. [arXiv:2404.18608](#), 2024.
- [3] J. El Gammal, R. Buscicchio, G. Nardini, and J. Torrado. Accelerating LISA inference with Gaussian processes. [arXiv:2503.21871](#), 2025.
- [4] Triangle Data Center. Triangle-BBH: Frequency-domain BBH TDI-2.0 response template and basic analysis tools. [GitHub repository](#), accessed July 2026.
- [5] L. Chang. task5-lisa-taiji: UCAS 2026 Task 5 LISA/Taiji data analysis repository. [GitHub repository](#), 2026.